

Projet de Compilation 2025-2026

L3 Informatique

Léo Pierrat

Mai 2026

Manuel d'utilisation

Le compilateur se construit avec :

`make`

L'exécutable produit est `bin/tpcc`. Il s'utilise de la façon suivante :

`./bin/tpcc [options] < monfichier.tpc`

Options disponibles :

Option	Description
<code>-t, --tree</code>	Affiche l'arbre abstrait sur la sortie standard
<code>-s, --syntabs</code>	Affiche toutes les tables des symboles
<code>-h, --help</code>	Affiche l'aide et termine

Codes de retour :

Code	Signification
0	Programme valide (même avec des avertissements)
1	Erreur lexicale ou syntaxique
2	Erreur sémantique

Description du compilateur

Analyse lexicale et syntaxique

L'analyse lexicale est réalisée avec Flex. Le lexeur gère les commentaires multi-lignes `/* ... */` via un état exclusif `COMM`, ce qui permet de continuer à compter les numéros de ligne à l'intérieur des commentaires. Les commentaires mono-ligne `// ...` sont ignorés par une règle simple. Les types `int` et `char` sont renvoyés comme token `TYPE` avec leur valeur, ce qui permet au parseur de les traiter uniformément.

L'analyse syntaxique est réalisée avec Bison. La grammaire implémente le langage TPC complet, y compris les types structure `struct X { ... }`. À chaque règle de grammaire correspond une action

qui construit un nœud de l'arbre abstrait via le module `tree`. Chaque nœud contient un label (type du nœud) et selon le cas un entier, un caractère ou une chaîne (identifiant ou opérateur).

Analyse sémantique

L'analyse sémantique est réalisée dans le module `sem.c`, par un parcours en profondeur de l'arbre abstrait.

Tables de symboles

Quatre tables de hachage sont utilisées (module `symb.c`) :

- `global_table` : variables globales
- `local_table` : variables locales à la fonction en cours (réinitialisée à chaque nouvelle fonction)
- `function_table` : noms et signatures des fonctions déclarées
- `struct_table` : noms des types struct déclarés

Chaque entrée de `function_table` encode, dans le champ `deplct`, le nombre de paramètres et le caractère void du type de retour : `deplct = param_count * 2 + is_void`. Cela permet de vérifier le nombre d'arguments et d'identifier les fonctions void sans structure de données supplémentaire.

Pré-passe

Avant le parcours principal, une pré-passe sur le nœud racine collecte tous les noms de fonctions. Cela permet de gérer les appels mutuellement récursifs et les appels à des fonctions déclarées plus loin dans le fichier source, sans fausse erreur. Les quatre fonctions built-in `putchar`, `getchar`, `putint`, `getint` sont également insérées dans `function_table` au démarrage.

Suivi des types struct (`struct_info`)

Le module `struct_info.c` maintient deux structures :

- `struct_defs` : liste chaînée de `StructDef`, chacune contenant la liste de ses champs (`FieldDef`) avec leur nom, leur indice et leur type (primitif ou struct imbriquée). La présence du type de champ (`struct_type`) permet de résoudre récursivement le type d'un accès enchaîné comme `b.a.x`.
- `var_types` : liste chaînée associant chaque variable de type struct à son nom de type. Cette liste est utilisée pour déterminer à l'analyse si un identifiant désigne une variable struct.

Pour éviter la pollution de portée entre fonctions (une variable `p` déclarée dans `f` ne doit pas être reconnue comme struct dans `g`), le pointeur de tête de `var_types` est sauvegardé au début de chaque fonction et restauré à la fin.

La fonction `get_node_struct_type(node)` infère récursivement le type struct d'une expression : elle retourne le nom du type si le nœud est un identifiant struct ou un accès de champ struct, et NULL sinon. Cette fonction est utilisée dans les vérifications de retour et d'affectation.

Erreurs sémantiques détectées

Erreurs sur la fonction `main` :

- `main` absente du programme
- `main` ne retourne pas `int` (retour `void`, `char` ou struct)

- `main` déclarée avec des paramètres

Déclarations :

- Variable utilisée sans être déclarée (locale ou globale)
- Fonction appelée sans être déclarée
- Type `struct X` utilisé alors que `X` n'a pas été déclaré (en variable, paramètre ou type de retour)
- Redéclaration d'une variable (locale ou globale)
- Redéclaration d'une fonction
- Redéclaration d'un type struct
- Redéclaration d'un champ dans un struct
- Redéclaration d'un paramètre
- Conflit entre une variable globale et une fonction de même nom

Appels de fonctions :

- Nombre d'arguments incorrect (trop peu ou trop)
- Résultat d'une fonction void utilisé dans une expression (affectation, condition, arithmétique, argument d'appel, instruction `return`)

Instructions `return` :

- `return expr` dans une fonction void
- `return` d'une valeur struct dans une fonction retournant un type primitif
- `return` d'une valeur non-struct dans une fonction retournant un struct

Opérations sur les structs :

- Struct utilisée comme opérande d'une opération arithmétique (+, -, *, /)
- Struct utilisée dans une comparaison (==, !=, <, >, <=, >=)
- Struct utilisée dans une opération logique (&&, ||, !)
- Struct utilisée comme condition d'un `if` ou d'un `while`
- Accès à un champ inexistant d'un struct
- Accès de champ sur une variable qui n'est pas de type struct
- Accès de champ enchaîné sur un champ de type primitif (`p.x.y` quand `x` est `int`)
- Accès de champ enchaîné invalide sur un champ struct (`b.a.z` quand `z` n'existe pas dans le type de `a`)

Affectations impliquant des structs :

- Affectation d'une valeur non-struct à une variable de type struct
- Affectation d'une variable struct à une variable de type primitif
- Affectation entre deux variables struct de types incompatibles
- Affectation d'un champ de type primitif à une variable struct (`p = q.x` quand `x` est `int`)
- Affectation d'un champ de type struct à une variable de type primitif (`n = b.a` quand `a` est un struct)

Avertissements

- Affectation d'une expression de type `int` à une variable de type `char`

Génération de code

Le code cible est de l'assembleur NASM x86-64 avec les conventions d'appel AMD64. Le fichier de sortie est `_anonymous.asm` par défaut (lecture sur l'entrée standard). En cas d'erreur sémantique, ce fichier est supprimé et le code de retour est 2.

Structure du programme généré. Le fichier assembleur contient :

- Un point d'entrée `_start` qui appelle `main` puis effectue le syscall `exit` avec le code de retour
- Les quatre fonctions built-in d'E/S écrites directement en assembleur
- Les fonctions du programme source
- Une section `.bss` pour les variables globales

Expressions. Les expressions sont évaluées par une approche à pile : chaque sous-expression empile son résultat, les opérateurs dépilent leurs opérandes dans `rax` et `r10` et empilent le résultat. La division utilise `idiv` après `xor rdx, rdx`. La multiplication utilise la forme à deux opérandes `imul rax, r10`.

Booléens. Les expressions booléennes (`&&`, `||`, `!`, `==`, `!=`, `<`, `>`, `<=`, `>=`) utilisent l'évaluation paresseuse (court-circuit) avec des labels. Chaque nœud booléen reçoit un `label_true` et un `label_false` et y saute directement sans passer par la pile.

Fonctions. Chaque fonction génère un prologue (`push rbp ; mov rbp, rsp ; sub rsp, N`) et un épilogue (`mov rsp, rbp ; pop rbp ; ret`). Les arguments sont passés dans les registres `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` et copiés sur la pile locale dans le prologue. Les variables locales sont allouées avec des déplacements négatifs par rapport à `rbp`.

Variables globales. Les variables globales sont allouées en section `.bss` via `resq`. Pour les variables de type struct, le nombre de quadwords réservés correspond au nombre de champs du struct. Les accès en lecture et écriture se font par label (`mov rax, [nom]` et `mov [nom], rax`).

Structs. Les champs d'un struct sont disposés consécutivement sur la pile (ou en `.bss`), séparés de 8 octets. L'indice de champ (calculé lors de l'enregistrement du struct) donne directement le décalage : `[rbp - offset - idx * 8]` pour une variable locale, `[label + idx * 8]` pour une variable globale.

Fonctions I/O built-in. Les quatre fonctions sont écrites directement en NASM en utilisant uniquement les syscalls Linux (`rax=0` pour `read`, `rax=1` pour `write`, `rax=60` pour `exit`). `putint` convertit l'entier en chaîne de caractères manuellement par divisions successives.

Difficultés rencontrées

Conventions AMD64. L'instruction `idiv` exige que le dividende soit dans la paire `rdx:rax` et crashe si `rdx` n'est pas initialisé à zéro avant l'appel. Il a fallu insérer systématiquement `xor rdx, rdx` avant chaque `idiv`. De même, l'alignement de la pile sur 16 octets avant un `call` est obligatoire sous Linux x86-64 : l'instruction `and rsp, -16` est insérée avant chaque appel de fonction pour garantir cet alignement.

Pollution de portée entre fonctions dans `var_types`. La liste `var_types` est globale et s'accroît au fil du parcours. Sans précaution, une variable struct déclarée dans une fonction `f` restait visible lors de l'analyse de la fonction suivante `g`. Cela causait de fausses erreurs : si `f` avait un paramètre `p` de type struct et que `g` avait un paramètre entier `p`, le `return p` ; de `g` était à tort signalé comme

retour de struct. La solution est de sauvegarder le pointeur de tête de `var_types` au début de chaque fonction et de libérer toutes les entrées ajoutées lors du nettoyage en fin de fonction.

Ordre de parcours de l'arbre et liste d'exclusion. La fonction `sem` utilise un parcours générique des enfants après le traitement du nœud courant. Certains nœuds (comme `T_FUNC`, `T_FCALL`, `T_ASSIGN`, `T_MEMBER_ACCESS`) gèrent eux-mêmes leur sous-arbre et doivent être exclus du parcours automatique pour éviter les double-visites ou les traitements dans le mauvais ordre. La liste d'exclusion a évolué au fur et à mesure que des erreurs de comptage ou de faux positifs apparaissaient.

Inférence de type pour les accès enchaînés. Détecter qu'un accès `b.a.x` est valide (ou non) nécessite de connaître le type du champ intermédiaire `a`. La structure `FieldDef` a été étendue pour stocker, en plus du nom et de l'indice du champ, le nom du type struct si le champ est lui-même un struct. La fonction récursive `get_node_struct_type` parcourt la chaîne d'accès et résout chaque niveau grâce à cette information, permettant de valider ou d'invalidier les accès imbriqués.

Faux positifs sur les affectations impliquant des accès de champ. L'extension de la détection du type struct au côté droit des affectations (via `get_node_struct_type`) a introduit un faux positif : `b.a = other_b.a` était signalé comme erreur car le côté gauche, de type `T_MEMBER_ACCESS`, avait `lhs_stype == NULL` tandis que le côté droit était reconnu comme struct. La correction consiste à ne déclencher la vérification « affectation d'un struct à un type de base » que lorsque le côté gauche est un identifiant simple (`T_IDENT`), et non un accès de champ.

IA.J'ai pas le temps de mettre un message sur l'ia mais j'aurai aimé en parler.